

Binary Search, Sorting, and Boundary Proofs

Binary search finds a boundary in an ordered or monotone search space. Its correctness depends on the meaning of the bounds and the update rule. Sorting creates useful order but has its own cost, stability, and memory implications. Learn both as reasoning tools rather than syntax patterns.

For: Candidates who repeatedly encounter off-by-one errors or use binary search by template without explaining why a candidate region can be discarded.

Suggested pace: 10 focused practice sessions | Arrays and complexity understood

What you will be able to do

- Implement lower and upper bounds with a consistent interval convention.
- Recognize monotone feasibility and search over answers safely.
- Explain sorting cost, stability, and the effect on original positions.
- Prove termination and test minimal and boundary cases.

Build the right mental model

Define the boundary

A lower bound is the first position whose value is at least the target; an upper bound is the first position greater than it. These remain useful even when the target is absent. Decide whether the interval is closed or half-open and keep all updates consistent.

Monotone predicates extend the method

Capacity or speed problems can have a false-then-true feasibility predicate. Search for the first feasible answer only after proving the monotonicity and establishing valid bounds. The cost includes every feasibility check, not just the logarithmic number of iterations.

Sorting changes the problem state

Sorting can enable pointer sweeps and grouping, but may mutate input or lose original order. Stable sorting preserves equal-key order, which matters for some contracts. Library time and auxiliary-memory guarantees vary, so cite the actual implementation model.

Study with evidence.

Use one clear lower-bound implementation until its invariant is natural. Draw the retained interval after each comparison. For answer search, test the feasibility function independently before combining it with binary search.

Sessions 1-3: Bound conventions

Implement exact search and lower bound with half-open intervals. Test empty input, one element, target below all values, target above all values, and duplicates. Explain what positions are known impossible after each update.

Deliverable and check: A trace proving the interval shrinks and the final position satisfies the boundary definition.

Sessions 4-5: Duplicates and derived queries

Use lower and upper bounds to find a target range or count occurrences. Preserve original indexes if sorting is introduced. Avoid special-case logic that contradicts the chosen interval convention.

Deliverable and check: Tests showing absent targets and repeated values produce correct ranges and counts.

Sessions 6-8: Search on an answer

Practice minimum capacity or speed with a monotone feasibility check. Derive low and high bounds from the input, use integer arithmetic for ceilings, and state whether the answer is guaranteed to exist.

Deliverable and check: A separately tested predicate and a complexity formula including its linear or other cost.

Sessions 9-10: Sorting trade-offs and mixed problems

Compare sorting with hashing and selection. Discuss stable order, comparator consistency, worst-case versus expected bounds, and output requirements. Solve an unseen problem that does not explicitly mention binary search.

Deliverable and check: A justified choice of ordering or monotonicity rather than a memorized problem label.

A repeatable practice session

Spend about 15 minutes recalling the previous topic without notes, 45 minutes on today's problem or lab, and 15 minutes explaining and checking the result. Record the failed assumption, not just the final answer. Revisit weak material after a few days and again the following week.

Apply it. Explain it.

Find the first value at least a target

For sorted values $[1,3,3,7]$ and target 3, the answer is index 1. For target 4, it is index 3. A boundary search should handle both without assuming the target exists.

A worked approach

1. Maintain a half-open candidate interval $[\text{left}, \text{right})$, initially $[0, 4)$. Values before left are known smaller than the target, and the answer can be right as an insertion position. Compute a midpoint inside the interval.
2. If the midpoint value is smaller than the target, move left to midpoint plus one. Otherwise move right to midpoint, retaining it as a possible first qualifying position. For target 3, equal values move the right boundary leftward.
3. Stop when left equals right. Verify the position is either the array length or a qualifying value, and that the preceding value, if any, is smaller. This postcondition is stronger than merely checking a few returned indexes.

Expected result

The same $O(\log n)$ -time, $O(1)$ -auxiliary-space boundary method supports insertion positions and duplicate ranges. Its correctness follows from retained candidates and a shrinking interval.

Verify the result

- Does an empty array return insertion position zero?
- Do targets below and above all values return zero and length?
- Do duplicates return the first qualifying index rather than an arbitrary match?

Extend the exercise

Replace array lookup with a shipping-capacity feasibility function. Prove that increasing capacity cannot require more days, then explain why total complexity is $O(n \log R)$ for answer range R rather than simply $O(\log n)$.

Make the learning stick.

Why does binary search sometimes loop forever?

An update may leave the interval unchanged, often because midpoint is retained on the wrong side under a mismatched convention. State the bounds precisely and prove each branch strictly reduces the candidate range.

Can I binary-search any boolean function?

No. The relevant truth values must have a monotone boundary over the searched domain, and the interval must bracket the answer appropriately. Without that property, discarding half the domain can remove valid answers.

Is sorting always $O(n \log n)$ and in-place?

Not universally. Algorithms and language libraries have different worst-case, expected, stability, and auxiliary-memory properties. Analyze the chosen implementation and include copies or index-pair storage required by the problem.

Avoid these traps

- Mixing closed and half-open bound updates.
- Using floating-point arithmetic where integer feasibility is required.
- Ignoring the cost of a feasibility check or input copy.

Ready for the next step?

- The search invariant and termination argument are explicit.
- Boundary and duplicate cases pass.
- Monotonicity is proved before searching an answer space.
- Sorting and memory costs match the actual implementation.

Reliable next reads

- [Python bisect](#)
- [CP-Algorithms: binary search](#)
- [Python sorting guide](#)

An independent preparation guide, not an employer-endorsed syllabus or a guarantee of an offer. Company examples are practice contexts, not verified past interview questions. Use only public, synthetic, or authorized material in exercises.